```
# A Python dictionary is a data structure that stores the value in key: value pairs.
# Values in a dictionary can be of any data type and can be duplicated, whereas keys can’t be repeated and must be immutable.

# A dictionary is an unordered, mutable collection that stores data in key-value pairs.
# It is defined using {} (curly brackets) and has keys that must be unique and immutable (e.g., strings, numbers, or tuples), w

# Python 3.6 and earlier: Unordered (implementation-dependent).
# From Python 3.7 Version onward, Python dictionary are Ordered.
# Dictionary keys are case sensitive: the same name but different cases of Key will be treated distinctly.
# Keys must be immutable: This means keys can be strings, numbers, or tuples but not lists.
# Keys must be unique: Duplicate keys are not allowed and any duplicate key will overwrite the previous value.
# Dictionary internally uses Hashing. Hence, operations like search, insert, delete can be performed in Constant Time.
```

```
# Creating a dictionary
student = {
    "name": "Alice",
    "age": 22,
    "grade": "A",
    "subjects": ["Math", "Science", "English"]
}

print(student)
```

```
{'name': 'Alice', 'age': 22, 'grade': 'A', 'subjects': ['Math', 'Science', 'English']}
```

```
# create dictionary using { }
d1 = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d1)

# create dictionary using dict() constructor
d2 = dict(a = "Geeks", b = "for", c = "Geeks")
print(d2)
```

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
{'a': 'Geeks', 'b': 'for', 'c': 'Geeks'}
```

```
student = {"name": "Alice", "age": 22, "grade": "A"}

# Accessing using square brackets []
print(student["name"])  # Output: Alice

# Accessing using get() method (preferred to avoid errors)
print(student.get("age"))  # Output: 22
print(student.get("city", "Not Found"))  # Output: Not Found (default value)
```

```
Alice
22
Not Found
```

```
# Get Key from Value in a Dictionary
# This returns a list because multiple keys may have the same value.
s = {1:"Govind",2:"Sahil",3:"Sanchit",4:"Minal",5:"Sahil"}

keys = [k for k, v in s.items() if v == "Sahil"]
print(keys)

#search for first value key
for key, val in s.items():
    if val == "Sahil":
        print(key)
        break  # Stop after finding the first match
```

```
[2, 5]
2
```

```
# Creating a dictionary
student = {"name": "Alice", "age": 22}

# Adding a new key-value pair
student["grade"] = "A"

# Updating an existing key
student["age"] = 23
```

```
print(student)
```

```
{'name': 'Alice', 'age': 23, 'grade': 'A'}
```

```
# The update() method is used to update multiple key-value pairs at once.
student.update({"city": "New York", "age": 24,"marks":90})
print(student)
```

```
{'name': 'Alice', 'age': 24, 'grade': 'A', 'city': 'New York', 'marks': 90}
```

```
# Using pop(key) – Removes a specific key and returns its value
student = {"name": "Alice", "age": 22, "grade": "A"}
print(student.pop("age"))
print(student)  # {'name': 'Alice', 'grade': 'A'}
```

```
22
{'name': 'Alice', 'grade': 'A'}
```

```
# Using popitem() – Removes the last key-value pair (for Python 3.7+)
student = {"name": "Alice", "age": 22, "grade": "A"}
print(student.popitem())
print(student)  # {'name': 'Alice', 'age': 22}
```

```
('grade', 'A')
{'name': 'Alice', 'age': 22}
```

```
student = {"name": "Alice", "age": 22, "grade": "A"}
del student["grade"]
print(student)  # {'name': 'Alice', 'age': 22}

# Delete entire dictionary
del student
```

```
{'name': 'Alice', 'age': 22}
```

```
student = {"name": "Alice", "age": 22, "grade": "A"}
student.clear()
print(student)  # {}
```

```
{}
```

```
# Iterating through keys
student = {"name": "Alice", "age": 22, "grade": "A"}
for key in student.keys():
    print(key)
```

```
name
age
grade
```

```
# Iterating through values
student = {"name": "Alice", "age": 22, "grade": "A"}
for value in student.values():
    print(value)
```

```
Alice
22
A
```

```
# Iterating through key-value pairs using items()
student = {"name": "Alice", "age": 22, "grade": "A"}
for key, value in student.items():
    print(key," ",value)
```

```
name   Alice
age   22
grade   A
```

```
# nested dictionary
d = {1: 'Geeks', 2: 'For',
```

```
        3: {'A': 'Welcome', 'B': 'To', 'C': 'Geeks'}}

print(d)
```

```
{1: 'Geeks', 2: 'For', 3: {'A': 'Welcome', 'B': 'To', 'C': 'Geeks'}}
```

```
# The setdefault() method is used to retrieve the value of a given key in a dictionary. If the key is not present, it inserts t
student = {"name": "Alice", "age": 22}

# Key 'age' exists, so it returns its value
value = student.setdefault("age", 25)
print(student)  # {'name': 'Alice', 'age': 22}
print(value)    # 22
```

```
{'name': 'Alice', 'age': 22}
22
```

```
student = {"name": "Alice", "age": 22}

# Key 'grade' does not exist, so it adds 'grade': 'A'
value = student.setdefault("grade", "A")

print(student)  # {'name': 'Alice', 'age': 22, 'grade': 'A'}
print(value)    # 'A'
```

```
{'name': 'Alice', 'age': 22, 'grade': 'A'}
A
```

```
student = {"name": "Alice", "age": 22}

# Key 'city' does not exist, so it adds 'city': None
value = student.setdefault("city")

print(student)  # {'name': 'Alice', 'age': 22, 'city': None}
print(value)    # None
```

```
{'name': 'Alice', 'age': 22, 'city': None}
None
```

```
# Creating a Dictionary from a List
numbers = [1, 2, 3, 4]
squares = {num: num**2 for num in numbers}
print(squares)
```

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

```
even_squares = {x: x**2 for x in range(1, 11) if x % 2 == 0}
print(even_squares)
```

```
{2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

```
num_status = {x: ("Even" if x % 2 == 0 else "Odd") for x in range(1, 6)}
print(num_status)
```

```
{1: 'Odd', 2: 'Even', 3: 'Odd', 4: 'Even', 5: 'Odd'}
```

```
#swapping key values in dictionary

original = {"a": 1, "b": 2, "c": 3}
swapped = {v: k for k, v in original.items()}
print(swapped)
```

```
{1: 'a', 2: 'b', 3: 'c'}
```

```
original = {'a': 1, 'b': {'x': 10, 'y': 20}}

# Shallow copy
shallow_copy = original.copy()

# Modify the nested dictionary
```

```
shallow_copy['b']['x'] = 100

print(original)       # {'a': 1, 'b': {'x': 100, 'y': 20}}
print(shallow_copy)   # {'a': 1, 'b': {'x': 100, 'y': 20}}

# ⚠️ Notice: Changes to the nested dictionary (b) reflect in both copies! This is because dict.copy() only copies top-level key
```

```
{'a': 1, 'b': {'x': 100, 'y': 20}}
{'a': 1, 'b': {'x': 100, 'y': 20}}
```

```
# A deep copy creates a completely independent copy, including nested dictionaries.
import copy

original = {'a': 1, 'b': {'x': 10, 'y': 20}}

# Deep copy
deep_copy = copy.deepcopy(original)

# Modify nested dictionary
deep_copy['b']['x'] = 100

print(original)   # {'a': 1, 'b': {'x': 10, 'y': 20}}
print(deep_copy)  # {'a': 1, 'b': {'x': 100, 'y': 20}}
```

```
{'a': 1, 'b': {'x': 10, 'y': 20}}
{'a': 1, 'b': {'x': 100, 'y': 20}}
```

```
#key exsistence
a = {"name":"Govind",1:[1,2,3],"age":24}

print("name" in a)
print("age" not in a)
print(24 in a.values())
```

```
True
False
True
```

```
#length
print(len(a))
```

```
3
```

```
#merging dictionary
# Using update() (Modifies First Dictionary)
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 20, 'c': 3}

dict1.update(dict2)

print(dict1)  # {'a': 1, 'b': 20, 'c': 3}
```

```
{'a': 1, 'b': 20, 'c': 3}
```

```
# Using | (Python 3.9+)
# This creates a new dictionary without modifying originals.
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 20, 'c': 3}

merged = dict1 | dict2

print(merged)  # {'a': 1, 'b': 20, 'c': 3}
print(dict1)
# ✅ Does NOT modify original dictionaries
```

```
{'a': 1, 'b': 20, 'c': 3}
{'a': 1, 'b': 2}
```

```
# Using {**dict1, **dict2} (Python 3.5+)
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 20, 'c': 3}

merged = {**dict1, **dict2}

print(merged)  # {'a': 1, 'b': 20, 'c': 3}
```

```
# ✅ Creates a new dictionary
# ❌ If keys overlap, dict2 values overwrite dict1 values
```

```
{'a': 1, 'b': 20, 'c': 3}
```

```
# Sorting by Keys (sorted() with dict.items())
# Sort a dictionary by keys (default behavior of sorted()):
my_dict = {'b': 3, 'a': 1, 'c': 2}

sorted_by_keys = dict(sorted(my_dict.items(), key = lambda item : item[0]))

print(sorted_by_keys)  # {'a': 1, 'b': 3, 'c': 2}
# ✅ Returns a new sorted dictionary
# ✅ Sorts in ascending order (default)
```

```
{'a': 1, 'b': 3, 'c': 2}
```

```
# Sorting by Values
# Sort a dictionary by values using lambda:
my_dict = {'b': 3, 'a': 1, 'c': 2}

sorted_by_values = dict(sorted(my_dict.items(), key=lambda item: item[1]))

print(sorted_by_values)  # {'a': 1, 'c': 2, 'b': 3}
# ✅ Sorts by values instead of keys

# Descending Order
# Use reverse=True to sort in descending order:
sorted_desc = dict(sorted(my_dict.items(), key=lambda item: item[1], reverse=True))
print(sorted_desc)  # {'b': 3, 'c': 2, 'a': 1}
```

```
{'a': 1, 'c': 2, 'b': 3}
{'b': 3, 'c': 2, 'a': 1}
```